

PROJECT DOCUMENTATION — 18 MAY 2026

AI Web Tester

Intelligent Website Quality Checker
powered by Claude AI + Playwright

AI BRAIN

Claude API

BROWSER

Playwright

BACKEND

FastAPI

FRONTEND

Next.js

DATABASE

SQLite

LANGUAGE

Python + JS

Built by: Biswa | Organisation: Webisdom

Date: 18 May 2026 | Model: claude-sonnet-4-6

What Is This Tool?

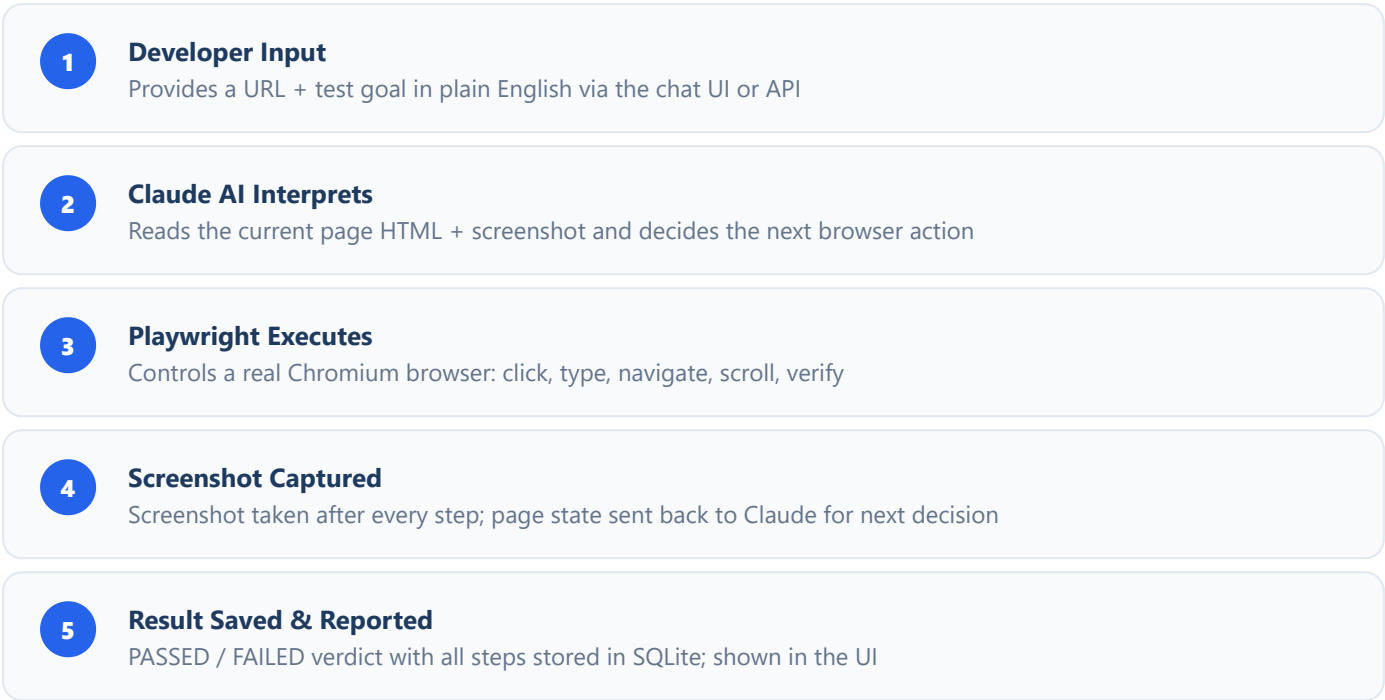
The **AI Web Tester** is an intelligent website quality-checking tool. Instead of writing test scripts manually, a developer simply describes a test goal in plain English. The AI reads the website, decides what browser actions to take, executes them on a real browser, and reports a **PASSED / FAILED** verdict with screenshots at every step.

Example: Developer types: *"Test the login flow on https://client-site.com"*
→ AI opens the site → finds login → fills credentials → clicks login → checks dashboard loads → Reports: **PASSED**

Why AI-Powered Testing?

Without AI	With AI Web Tester
Write test scripts manually	Describe goal in plain English
Works only on specific sites	Works on ANY website
Breaks when UI changes	AI adapts to current page structure
Requires coding knowledge	Anyone on the team can use it
Static test cases	Dynamic, intelligent decision-making

Core Architecture Flow



Project Structure

Location: C:\Users\biswa\OneDrive\Desktop\AI assistant tool\ai-web-tester\

```
ai-web-tester/
├── backend/           ← FastAPI + Claude AI + Playwright
│   ├── main.py       ← FastAPI server & API endpoints
│   ├── agent.py      ← Claude AI agent loop (brain)
│   ├── browser.py    ← Playwright browser controller
│   ├── database.py   ← SQLite database layer
│   ├── show_reports.py ← CLI report viewer
│   ├── requirements.txt
│   ├── .env          ← ANTHROPIC_API_KEY
│   ├── web_tester.db ← SQLite database
│   └── screenshots/  ← PNG screenshots per step
├── frontend/         ← Next.js Chat UI
│   ├── pages/
│   │   ├── index.js  ← Main testing interface
│   │   └── reports.js ← Full reports viewer
│   ├── components/
│   │   ├── TestResult.js ← Pass/fail display + screenshots
│   │   └── HistoryPanel.js ← Recent tests sidebar
│   └── package.json
└── README.md
```

Tech Stack

Layer	Technology	Version	Purpose
AI Reasoning	Claude API (Anthropic)	claude-sonnet-4-6	Interprets goals, decides browser actions
Browser Control	Playwright (Python)	1.59.0	Controls real Chromium browser
Backend API	FastAPI + Uvicorn	0.136.1	REST API server
Frontend UI	Next.js + Tailwind CSS	14.2.5	Chat interface & reports
Database	SQLite	built-in	Stores all test runs & steps
AI SDK	anthropic (Python)	0.102.0	Claude API client

Backend — Key Files Explained

main.py — FastAPI Server

The entry point of the backend. Sets up the REST API with CORS, mounts the screenshots directory as a static file server, and exposes these endpoints:

Method	Endpoint	Description
POST	/test	Run a new AI-powered test. Accepts url + goal + optional credentials
GET	/tests	List all past test runs from SQLite (newest first)
GET	/tests/{id}	Get full detail of one test run including all steps
DELETE	/tests/{id}	Delete a test run and its steps
GET	/health	Health check — also confirms API key is configured
GET	/screenshots/{file}	Serves screenshot PNG files

Key design decision: uses **absolute paths** (via `__file__`) for the database and screenshots folder so the server can be started from any directory.

agent.py — Claude AI Agent Loop

This is the "brain" of the tool. The agent loop works as follows:

```
for step in range(max_steps=15):
    page_html = get current page HTML (stripped of scripts/styles)
    screenshot = take screenshot of current state

    → Send to Claude: goal + page HTML + screenshot (multimodal)
    ← Claude returns JSON: { action, target, value, reasoning, is_complete, verdict }

    → Execute the action via Playwright (click / type / check / scroll ...)
    → Take new screenshot
    → Save step result

    if is_complete: return PASSED / FAILED verdict
```

The system prompt instructs Claude to respond **only with a JSON object** — no markdown, no extra text — making parsing reliable. If Claude's response can't be parsed, the agent marks the test as FAILED with a clear error.

browser.py — Playwright Controller

Wraps Playwright's async API into simple methods. Each method tries multiple strategies to find elements:

Method	What It Does
<code>go_to(url)</code>	Navigates to URL, auto-adds https:// if missing

<code>click(selector)</code>	Tries CSS selector → text match → role (button/link) as fallback
<code>type(selector, text)</code>	Fills input field using <code>fill()</code> or <code>type()</code> as fallback
<code>check(selector, value)</code>	Verifies element exists or page contains expected text
<code>scroll(direction)</code>	Scrolls up or down 600px via JavaScript
<code>screenshot(name)</code>	Captures PNG with timestamp, saves to backend/screenshots/
<code>get_page_content()</code>	Returns cleaned HTML (scripts/styles stripped) for Claude

database.py — SQLite Layer

Two tables are created automatically on startup:

```
test_runs:
  id (TEXT PK) | url | goal | status | final_verdict | duration
  total_steps | passed_steps | failed_steps | error | created_at

test_steps:
  id (INT PK) | test_run_id (FK) | step_number | action | target
  value | reasoning | status | error_message | screenshot_path | created_at
```

Frontend — Next.js Chat UI

pages/index.js — Main Interface

Chat-style UI with:

- URL input field
- Test goal textarea + example chips
- Optional credentials (username/password)
- Animated loading state while AI runs
- Inline test result display
- Link to Reports page
- History sidebar panel

pages/reports.js — Reports Viewer

Full-page database viewer:

- Stats bar: Total / Passed / Failed / Errors
- Scrollable run list on the left
- Detail panel on the right
- Full steps table with action/target/status/reasoning
- All screenshots displayed inline
- Delete button per run

components/TestResult.js

Displays after a test completes:

- Verdict banner (green=PASSED, red=FAILED)
- Step count and duration
- Expandable step-by-step list
- Action chips (colour-coded by type)
- Screenshot per step when expanded
- Detailed error message if test failed

components/HistoryPanel.js

Sidebar history from the database:

- Fetches GET /tests on load
- Shows URL, goal, verdict, steps, duration
- Click any run to reload it into the main view
- Delete button per row

Frontend → Backend Communication

```
// Run a test
POST http://localhost:8000/test
Body: { "url": "https://site.com", "goal": "Test login flow", "credentials": {"username": "...", "password": "..."} }

// Get all past runs
GET http://localhost:8000/tests?limit=20

// Get one run with all steps
GET http://localhost:8000/tests/{test_id}
```

The Next.js config proxies `/api/*` to the backend via `next.config.js` rewrites. Direct calls use `http://localhost:8000`.

Bugs Found & Fixed Today

Bug 1 — Tool returned ERROR with no details FIXED

Root cause: `uvicorn.run(..., reload=True)` spawns a child subprocess for file-watching. That child process couldn't reliably find the `.env` file, so `ANTHROPIC_API_KEY` was never loaded. The agent silently failed with an empty exception message.

Fix: Changed `reload=True` → `reload=False` in `main.py`. The server now runs in a single direct process that always loads `.env` correctly.

Bug 2 — Empty error message swallowed silently FIXED

Root cause: The agent caught all exceptions with `str(exc)`. If the exception had no message (e.g. bare `Exception()`), this returned `""` — so the UI showed ERROR with zero details.

Fix: Added fallback: `f"{type(exc).__name__}: {traceback.format_exc().splitlines()[-1]}"` so there is always a meaningful error string. Also added Python `logging` to print the full traceback to the server console.

Bug 3 — finally block could override original exception FIXED

Root cause: `await browser.stop()` was called in a bare `finally` block. If `stop()` itself raised, Python discards the original exception and raises the new one instead.

Fix: Wrapped `browser.stop()` in its own `try/except` inside the `finally` block.

Bug 4 — Frontend showed no error details when error was empty FIXED

Root cause: The error block in `TestResult.js` used `{error && ...}`. An empty string `""` is falsy in JavaScript, so the error section never rendered.

Fix: Changed condition to `{{(final_verdict === "ERROR" || error) && ...}}` with a fallback message: "Check that the backend is running and ANTHROPIC_API_KEY is set in backend/.env"

Bug 5 — Database / screenshots in wrong directory FIXED

Root cause: `Database(db_path="web_tester.db")` used a relative path. Running `python main.py` from a different directory would create the database there. OneDrive sync also caused the file to appear empty in DB Browser.

Fix: All paths now use `os.path.dirname(os.path.abspath(__file__))` so the database and screenshots always land in the `backend/` folder regardless of where the server is started.

Setup & How to Run

One-Time Setup (already done)

Step	Command	Status
Create Python venv	<code>python -m venv venv</code>	DONE
Install Python deps	<code>pip install fastapi uvicorn anthropic playwright python-dotenv</code>	DONE
Install Chromium browser	<code>playwright install chromium</code>	DONE
Set API key	Edit <code>backend/.env</code> → <code>ANTHROPIC_API_KEY=sk-ant-...</code>	DONE
Install Node deps	<code>cd frontend && npm install</code>	DONE

Every Day — Start the Tool

Terminal 1 — Start Backend:

```
cd "C:\Users\biswa\OneDrive\Desktop\AI assistant tool\ai-web-tester\backend"
venv\Scripts\activate
python main.py
# Server starts at http://localhost:8000
```

Terminal 2 — Start Frontend:

```
cd "C:\Users\biswa\OneDrive\Desktop\AI assistant tool\ai-web-tester\frontend"
npm run dev
# UI opens at http://localhost:3000
```

Using the Tool

- 1

Open `http://localhost:3000`
The chat interface loads in your browser
- 2

Enter a website URL
e.g. `https://example.com` or any client website
- 3

Describe the test goal
e.g. "Test the login flow", "Verify the contact form works", "Check search returns results"
- 4

(Optional) Add login credentials
Expand the credentials section and enter username/password if login is needed
- Click Run Test**

AI controls the browser. Results appear with PASSED/FAILED verdict and screenshots

6

View full reports at </reports>

All past test runs with step-by-step breakdown and screenshots from SQLite

Viewing the SQLite Database

Method	How
In-app Reports page	Go to http://localhost:3000/reports — best option, no extra tools needed
CLI script	<code>cd backend && venv\Scripts\python show_reports.py</code>
DB Browser for SQLite	Download from sqlitebrowser.org → Open <code>backend\web_tester.db</code>
VS Code Extension	Install "SQLite Viewer" extension → open the .db file directly
REST API	http://localhost:8000/tests in browser while backend runs

Database Schema & Sample Data

Table: test_runs

Column	Type	Description
id	TEXT (PK)	UUID — unique identifier for each test run
url	TEXT	Website URL that was tested
goal	TEXT	Plain-English test goal provided by the developer
status	TEXT	completed / error / max_steps_reached
final_verdict	TEXT	PASSED / FAILED / ERROR
duration	REAL	Total time taken in seconds
total_steps	INTEGER	Number of browser actions taken
passed_steps	INTEGER	Steps that executed successfully
failed_steps	INTEGER	Steps that encountered errors
error	TEXT	Error message if status is error
created_at	TEXT	ISO datetime when the test ran

Table: test_steps

Column	Type	Description
id	INTEGER (PK)	Auto-increment step ID
test_run_id	TEXT (FK)	Links to test_runs.id
step_number	INTEGER	Step order (1, 2, 3 ...)
action	TEXT	go_to / click / type / check / scroll / wait / done
target	TEXT	CSS selector or text Claude chose as the target
value	TEXT	Text value for type/select actions
reasoning	TEXT	Claude's explanation for choosing this action
status	TEXT	passed / failed
error_message	TEXT	Playwright error if step failed
screenshot_path	TEXT	Absolute path to the PNG screenshot for this step
created_at	TEXT	ISO datetime when step was recorded

URL	Goal	Verdict	Steps	Duration
example.com	Check if the page loads	PASSED	1	6.7s
example.com	Verify homepage loads with heading	PASSED	1	6.9s
google.com	Check if search functionality works	PASSED	4	32s
google.com	Search for IPL news, save top 5 links	FAILED	7	126s
various	Early test runs (before bug fix)	ERROR	0	<0.1s